iOS In-App Purchase in Flutter

~ by Tushar

What is In-App Purchase (IAP)?

In-App Purchase is a service provided by Apple that allows you to sell digital content, services, or subscriptions directly within your Flutter app. Instead of asking a user to pull out their credit card and type in details, they simply "double-click to pay" using the payment method already linked to their Apple ID.

Why do we need iOS IAP?

If you are developing for iOS, IAP isn't just a "feature"—it is often a **requirement**.

- **User Trust:** Users feel safer paying through Apple than giving their card details to a third-party app.
- **Subscription Management:** Apple provides a native UI for users to manage, cancel, or upgrade their subscriptions in one place.

The "No Third-Party" Rule: Why Apple Rejects Apps

A common question developers ask is: *"Can't I just use Stripe or PayPal to avoid Apple's 15–30% commission?"*

The short answer is No. If your app sells **digital goods** (like premium features, ad-removal, or subscriptions), Apple's App Store Review Guideline 3.1.1 strictly mandates the use of their native IAP system.

If you use a third-party payment link for digital goods, Apple will reject your app because:

1. **Safety & Privacy:** Apple wants to ensure they can mediate disputes and process refunds if something goes wrong.
2. **Business Model:** This is how Apple monetizes the App Store platform.
3. **Consistency:** They want a uniform checkout experience across all iOS apps. *Note: You can only use third-party payments (like Stripe) if you are selling **physical goods** (like a t-shirt or food delivery).*

Prerequisites:

Before touching any code, ensure you have the following "Starter Kit" ready:

- **Hardware:** A Mac with Xcode installed and a **physical iOS device** (In-app purchases cannot be fully tested on a Simulator).
- **Account:** An active Apple Developer Program membership and access to App Store Connect.
- **Flutter Package:** We will be using the official `in_app_purchase` plugin.

Step 1: Xcode Configuration

Open your project's iOS workspace in Xcode (`ios/Runner.xcworkspace`):

1. Select **Runner** in the Project Navigator.
2. Go to the **Signing & Capabilities** tab.
3. Click the **+ Capability** button and search for **In-App Purchase**. Add it.

- *Note: If you don't do this, your app won't be able to communicate with the App Store protocols.*

Step 2: Setting up App Store Connect

Navigate to your app in App Store Connect and scroll down to **Subscriptions**.

1. **Create a Subscription Group:** Think of this as the "container" for your levels (e.g., "Premium_Tier").
2. **Configure the Group:**
   - Add a **Localization** name for the group.
   - **Grace Period:** If you enable this, remember Apple only allows specific durations: **3, 16, or 28 days**.
3. **Create Individual Subscriptions:** (e.g., Monthly, Half-Yearly, Yearly).
   - **Product ID:** Create a clear, unique ID (e.g., `com.yourapp.monthly_premium`).
   - **Pricing:** Here is a catch—Apple uses **Price Tiers**. If your specific price (e.g., $9.87) isn't in their preset list, you'll have to choose the closest available tier (like $9.99).
   - **Availability:** Choose the countries where your subscription will be live.
   - **Review Metadata:** You **must** upload a screenshot of your app's subscription page (the "Paywall"). If this isn't provided, your status won't move to **"Ready to Submit,"** and the products won't fetch in your code.
4. **Introductory Offers (Free Trials & Discounts):**
   - Inside the specific subscription (e.g., Monthly), click the **"+"** next to **Subscription Prices** and select **Set Up Introductory Offer**.
   - **Free Trial:** You can offer a period (e.g., 3 days, 1 week, 1 month) where the user isn't charged.
   - **Pay As You Go:** Users pay a reduced price for a few periods (e.g., $1.99/month for the first 3 months) before the standard price kicks in.
   - **Pay Up Front:** Users pay a one-time discounted price for a specific duration (e.g., $9.99 for the first 6 months).
   - *Note: Only new subscribers are typically eligible for these, and Apple handles this eligibility check automatically.*

Step 3: Local Testing with StoreKit Configuration

If your Paid Agreement isn't fully processed yet, or if you just want to test locally without hitting Apple's servers, use a **StoreKit Configuration File**.

1. **Create the File:** In Xcode, Right-click your project root -> New File -> Search for **"StoreKit Configuration File."**
2. **Syncing:** It can automatically fetch your products from App Store Connect if you're logged in.
3. **The Scheme Catch:** * Go to **Product > Scheme > Edit Scheme**.
    - Under **Run > Options**, select your new file in the "StoreKit Configuration" dropdown.
    - **CRITICAL:** When you are ready to deploy to **TestFlight** or **Production**, you must set this back to **"None."** If you leave the StoreKit file active in a build, the purchase flow will fail for real users.

Step 4: Live Fetching (The "Paid Agreement" Way)

Once your **Paid Applications Agreement** is active in App Store Connect (and you've provided your bank/tax info), the process becomes simpler. You can fetch products directly from Apple's servers using the `in_app_purchase` package without needing the local StoreKit file for every test.

Code Part :

- I have used in_app_purchase package with version :
- I have used RIVERPOD in my flutter app so the code can be vary based on state management, coding style and other aspects.
- I have broken this down into the **Service Layer** (Logic), the **Riverpod Layer** (State Management), and the **UI Implementation**.

In_app_purchase.dart :

```
import 'dart:async';
import 'dart:developer';
import 'package:in_app_purchase/in_app_purchase.dart';

class InAppPurchaseService {
```

```
InAppPurchaseService._();
static final InAppPurchaseService instance = InAppPurchaseService._();

static final InAppPurchase _iap = InAppPurchase.instance;
StreamSubscription<List<PurchaseDetails>>? _subscription;

// Cache to track processed IDs and avoid redundant backend calls
final Set<String> _handledPurchaseIds = {};

static const Set<String> iapSubscriptionIds = {
  'monthly_subscription',
  'quarterly_subscription',
  'half_year_subscription',
  'yearly_subscription',
};

/// Initializes the IAP connection and checks if the store is available.
Future<bool> initialize() async {
  final available = await _iap.isAvailable();
  log('IAP Availability: $available');
  return available;
}

/// Queries App Store Connect for product information based on defined IDs.
Future<List<ProductDetails>> fetchProducts() async {
  final response = await _iap.queryProductDetails(iapSubscriptionIds);

  if (response.error != null) {
    throw Exception('Product Query Error: ${response.error!.message}');
  }

  if (response.productDetails.isEmpty) {
    log('No products found. Ensure products are "Ready to Submit" in App Store Connect.');
  }

  return response.productDetails;
}

/// Launches the native iOS payment sheet for a specific product.
Future<void> buySubscription(ProductDetails product) async {
  final param = PurchaseParam(productDetails: product);
  await _iap.buyNonConsumable(purchaseParam: param);
}
```

```
/// Sets up the stream listener to handle purchase updates (purchased, restored, etc).
void listenToPurchases({
  required void Function(PurchaseDetails purchase) onVerified,
  required void Function(PurchaseDetails purchase) onError,
}) {
  _subscription ??= _iap.purchaseStream.listen(
    (purchases) async {
      for (final purchase in purchases) {
        final purchaseId = purchase.purchaseID ?? '';

        // Skip if this specific transaction instance was already handled
        if (_handledPurchaseIds.contains(purchaseId)) continue;

        switch (purchase.status) {
          case PurchaseStatus.purchased:
          case PurchaseStatus.restored:
            _handledPurchaseIds.add(purchaseId);
            // Complete the transaction with Apple before notifying the UI
            await _complete(purchase);
            onVerified(purchase);
            break;

          case PurchaseStatus.pending:
            log('Purchase Status: Pending');
            break;

          case PurchaseStatus.error:
            onError(purchase);
            break;

          case PurchaseStatus.canceled:
            log('Purchase Status: Canceled');
            break;
        }
      }
    },
    onError: (e) => log('Purchase Stream Error: $e'),
  );
}

/// Signals to the App Store that the delivery of the product is successful.
Future<void> _complete(PurchaseDetails purchase) async {
```

```
    if (purchase.pendingCompletePurchase) {
      await _iap.completePurchase(purchase);
    }
  }

  /// Triggers the native Apple restore flow for existing subscriptions.
  Future<void> restorePurchases() async {
    await _iap.restorePurchases();
  }

  /// Clears the local cache of handled IDs. Useful for local testing.
  void clearHandledPurchases() {
    _handledPurchaseIds.clear();
  }

  /// Cancels the stream subscription when the service is no longer needed.
  void dispose() {
    _subscription?.cancel();
    _subscription = null;
    _handledPurchaseIds.clear();
  }
}
```

Riverpod State Management :

```
// Provider for the IAP Service instance
final inAppPurchaseServiceProvider = Provider<InAppPurchaseService>((ref) {
  return InAppPurchaseService.instance;
});

// FutureProvider to fetch and cache product details
final iapProductsProvider = FutureProvider<List<ProductDetails>>((ref) async {
  final iap = ref.read(inAppPurchaseServiceProvider);
  await iap.initialize();
  return iap.fetchProducts();
});

// Listener to handle the verification logic when a purchase event occurs
final iapListenerProvider = Provider<void>((ref) {
  final iap = ref.read(inAppPurchaseServiceProvider);

  iap.listenToPurchases(
```

```
    onVerified: (purchase) async {
      final purchaseState = ref.read(iapPurchaseContextProvider);
      if (purchaseState.context == null || !purchaseState.context!.mounted) return;

      try {
        // Refresh the iOS receipt data for the latest transaction info
        final iosAddition = InAppPurchase.instance
            .getPlatformAddition<InAppPurchaseStoreKitPlatformAddition>();
        await iosAddition.refreshPurchaseVerificationData();

        final rawReceipt = purchase.verificationData.serverVerificationData;

        // Backend Verification Logic
        // 1. Send rawReceipt to your server
        // 2. Server validates with Apple's /verifyReceipt or App Store Server API
        // 3. Update local user state based on server response

        // If success, navigate to success screen
        ref.read(iapLoadingProvider.notifier).hideLoading();
        purchaseState.context!.pushNamed(RouteNames.paymentSuccessScreen);

      } catch (e) {
        log('Verification Error: $e');
        ref.read(iapLoadingProvider.notifier).hideLoading();
      }
    },
    onError: (purchase) {
      ref.read(iapLoadingProvider.notifier).hideLoading();
    },
  );
});
```

Riverpod State Management :

```
/// Handles the iOS specific purchase flow including loading indicators
Future<void> _handleIOSPurchase(dynamic plan) async {
  final loading = ref.read(iapLoadingProvider.notifier);
  loading.showLoading('Processing payment...');
```

```
  try {
    final productId = _getProductIdFromDuration(plan.duration);
    final products = await ref.read(iapProductsProvider.future);
    final product = products.where((p) => p.id == productId).firstOrNull;

    if (product == null) {
      loading.hideLoading();
      AppSnackBar.showToast(message: 'Product not available');
      return;
    }

    // Set context so the listener knows where to navigate after verification
    ref.read(iapPurchaseContextProvider.notifier).state = IAPPurchaseState(
      context: context,
      planName: plan.displayName,
      planPrice: plan.priceUSD,
    );

    // Clear previous transaction IDs to ensure fresh processing
    ref.read(inAppPurchaseServiceProvider).clearHandledPurchases();

    // Launch the App Store Payment Sheet
    await ref.read(inAppPurchaseServiceProvider).buySubscription(product);

  } catch (e) {
    loading.hideLoading();
    log('Purchase Initiation Error: $e');
  }
}
```

Summary of what these functions do:

`initialize()`: Checks if the device allows in-app purchases (Parental controls/Region).

`fetchProducts()`: Reaches out to Apple to get the latest prices and titles you configured in App Store Connect.

`buySubscription()`: Opens the "Double-click to Pay" system dialog.

`listenToPurchases()`: The most important part; it waits for Apple to say "Payment Success" and then triggers your logic to unlock content.

`_complete()`: Essential for iOS. It tells Apple you've acknowledged the purchase so they can close the transaction and pay you.

Backend Integration & Receipt Validation :

Once the iOS payment sheet confirms a successful purchase, you receive a `receiptData` token. This is the **Source of Truth** for the transaction.

1. **Transmission:** Your Flutter app sends this `receiptData` (and the environment type: sandbox or production) to your backend.
2. **Verification:** Your backend acts as a proxy, sending this token to Apple's verification servers.
3. **Decoding:** The receipt contains encoded information (which can be decoded like a JWT) including the purchase date, expiration date, and product ID.
4. **Database Sync:** Your backend saves these details and maps the user's account to the correct "Subscription Plan" logic.
5. **Status Delivery:** The frontend then fetches the user's plan status from your own database to unlock premium features.

Testing with Sandbox Accounts :

Testing IAP requires a **Sandbox Tester** account. You cannot use a standard Apple ID to test "fake" money transactions.

Creating a Sandbox User

1. Go to **App Store Connect** > **Users and Access** > **Sandbox**.
2. Create a new user using a "fake" email address (e.g., `yourapp_test01@gmail.com`).
    - **Pro Tip:** This email should not be a registered Apple ID. Use a consistent naming convention, as you will likely need multiple accounts to test different scenarios (like switching from Monthly to Yearly).
3. On your **physical iOS device**, go to **Settings > App Store > Sandbox Account** (or Developer settings) and sign in with these credentials.

Sandbox Subscription Durations

To make testing efficient, Apple "accelerates" time in the Sandbox environment. A subscription that normally lasts a month will expire in minutes so you can test renewal logic quickly.

1 Week becomes **3 Minutes**

1 Month becomes **5 Minutes**

2 Months becomes **10 Minutes**

3 Months becomes **15 Minutes**

6 Months becomes **30 Minutes**

1 Year becomes **1 Hour**

Note: After a few auto-renewals (usually 5–6 times), the subscription will stop renewing automatically to prevent infinite loops during testing.

Advanced Testing Scenarios :

You can manage your active sandbox subscriptions directly on the device (**Settings > App Store > Sandbox Account > Manage**). From here, you should test:

- **Cancellations:** Cancel the subscription and ensure your app/backend reflects the "Expired" or "Cancelled" status after the sandbox time elapses.
- **Plan Changes:** Upgrade or downgrade between subscription levels in your group and verify the backend updates the permissions correctly.
- **Grace Period & Webhooks:** If you have Server-to-Server notifications (Webhooks) set up, Apple will notify your backend the moment a renewal fails or a user cancels. This is essential for keeping your UI in sync without requiring the user to open the app.
- **Free Trials:** Ensure the "Introductory Price" or "Free Trial" logic only triggers for eligible accounts and that the backend correctly identifies the `isTrial` flag.